CS2323: Computer Architecture

Homework-4: Processor Design

MA24BTECH11022

Q1. A six-stage pipelined processor has delays of 130, 125, 110, 132, 135, and 115 picoseconds for each pipeline stage. The registers used between pipeline stages have a delay of 12 picoseconds each. [3 + 2 marks]

a. Find the total time to execute 1000 independent instructions on this pipelined processor, assuming there are no pipeline stalls/hazards.
Maximum delay among the 6 stages is 135 ps. Also the registers used between pipeline stages have a delay of 12 ps, so time period of a clock cycle $T = 135 + 12 = 147$ ps.
For a 6 stage pipeline, the total cycles needed for n instructions is $6 + n - 1$. Hence the total execution time $= T \times (5 + n) = 147 \times 1005$ ps $= 147.735$ nanoseconds.

b. Find the total time to execute 1000 independent instructions on this pipelined processor, assuming that 20% of instructions incur a 1-cycle stall.
Ignoring extra cycles due to stalling, we have 1005 cycles. Since 20% of the 1000 instructions incur a 1-cycle stall, $0.20 \times 1000 = 200$ stall cycles are incurred on top of the 1005 cycles. So the total execution time $= T \times (1005 + 200) = 147 \times 1205$ ps $= 177.135$ nanoseconds.

Q2. Consider a 5-stage pipelined processor P1 with a total latency of 800 ps, equally divided between the pipeline stages. P2 is an improved version of P1, which improves the ALU by supporting MULT instruction. Including the MULT instruction increases the ALU latency by 250 ps, with the latency of other stages being unchanged. Presence of MULT instruction also reduces the total number of instructions by 10% (as we can directly use MULT instead of realizing it through repeated ADD). Answer the following: [2 + 2 + 1 marks]

a. What is the minimum required clock period for processors P1 and P2?
For pipelined processors, it is assumed that each stage takes approximately same time, so for P1, the minimum required clock period $= 800/5 = 160$ ps.
Introduction of MULT increases the ALU latency, so the third stage (EX) will take $160 + 250 = 410$ ps whereas rest of the 4 stages will take 160 ps. Since the maximum delay is 410 ps, the minimum required clock period for P2 is 410 ps.

b. Which processor (P1 or P2) would execute a given code in a smaller time?
Suppose the given code has N instructions. Then P1 takes $(5+N-1)\times 160 = 160N+640$ ps to execute the code. Since introduction of MULT also reduces the number of instructions by 10%, P2 takes $(5 + 0.9N - 1) \times 410 = 369N + 1640$ ps. Now observe that $369N + 1640 > 160N + 640$ iff $209N > -1000$. Since N is obviously non-negative, for our purposes $369N + 1640 > 160N + 640$, hence P1 takes a smaller time for executing a piece of code.

c. If a piece of code has 5000 instructions on P1, find the total execution time of this code on P1 and P2. Execution time for P1 $= (5 + 5000 - 1) \times 160 = 800.64$ ns. The number of instructions to be executed in P2 $= 0.9 \times 5000 = 4500$. So execution time for $P2 = (5 + 4500 - 1) \times 410 = 1846.64$ ns.

Q3. Consider the code given below:

```
add x14, x12, x11
add x15, x14, x12
ld x13, 8(x13)
ld x12, 0(x14)
and x13, x15, x13
ld x11, 4(x13)
sd x13, 0(x15)
```

a. Insert NOPs for correct execution (assuming no forwarding or hazard detection is implemented). [3 marks]
For convenience, number the lines of the given code as `L1` to `L7`. Without hazard detection or forwarding, an instruction reading a register in ID (in the second half of the cycle) that was written by a previous instruction in WB (in the first half of the cycle) requires a gap of 2 instructions (or NOPs) if it immediately follows the producer. The hazards are:

1. L1-L2 (x14):
   L2 tries to read x14 in ID, but L1 writes x14 in WB (cycle 5). Thus, L2 must be delayed by 2 NOPs inserted between L1 and L2.
2. L3-L5 (x13):
   L3 writes x13 in WB. There is only 1 instruction (L4) between L3 and L5. To allow L5 to read the updated x13 in ID in cycle 9, 1 NOP must be inserted before L5.
3. L5-L6 (x13):
   L5 writes x13 in WB. L6 reads x13 in ID right after L5. Hence, L6 must be delayed by 2 NOPs inserted between L5 and L6.

After taking these into account, all other dependencies (L1-L4, L2-L5, L5-L7) have sufficient cycles between write-back and read. The final code with NOPs inserted is:

```
add x14, x12, x11
nop
nop
add x15, x14, x12
ld x13, 8(x13)
ld x12, 0(x14)
nop
and x13, x15, x13
nop
nop
ld x11, 4(x13)
sd x13, 0(x15)
```

b. Insert NOPs for correct execution (assuming data forwarding without hazard detection is implemented). [2 marks]
The (possible) hazards are:

1. L1-L2:
   L2 needs x14 in its EX stage. The result of L1 is available after its EX stage and can be forwarded directly to L2's EX stage. So no NOPs are needed.
2. L2-L5:
   By the time L5 reaches EX, the result of L2 is already available and can be forwarded/read. So no NOPs are needed.
3. L3-L5:
   The loaded value is available after L3's MEM stage. Since L4 is between L3 and L5, MEM/WB forwarding passes the loaded value to L5's EX stage in time. So no NOPs are needed.
4. L5-L6:
   L6 uses x13 as its address in EX. The ALU result from L5 can be forwarded directly to L6's EX stage. So no NOPs are needed.
5. L5-L7:
   There is L6 between them, so the value is available in time and can be forwarded to the store. So no NOPs are needed.

No NOPs are needed in the code, so it remains the same.

Q4. Consider the code given below:

```
    add x14, x12, x11
L2: beq x15, x14, L1
    ld x13, 8(x13)
    beq x12, x13, L2
L1: and x13, x15, x13
    ld x11, 4(x13)
    sd x13, 0(x15)
```

a. Insert NOPs for correct execution (assuming no forwarding or hazard detection is implemented). [3 marks]
Without forwarding, any dependent instruction immediately following a producer requires 2 NOPs. The hazards are:

1. L1-L2 (x14):
   L2 reads x14 in ID, but the first instruction writes x14 in WB in cycle 5. Hence, 2 NOPs are required before L2.

2. L3-L4 (x13):
   L3 (ld) writes x13 in WB. L4 (beq) reads x13 in ID immediately after L3. Hence, 2 NOPs are required between L3 and L4.
3. L5-L6 (x13):
   L5 (and) writes x13 in WB. L6 (ld) reads x13 in ID immediately after L5. Hence, 2 NOPs are required between L5 and L6.

Therefore, the final code is:

```
        add x14, x12, x11
        nop
        nop
    L2: beq x15, x14, L1
        ld x13, 8(x13)
        nop
        nop
        beq x12, x13, L2
    L1: and x13, x15, x13
        nop
        nop
        ld x11, 4(x13)
        sd x13, 0(x15)
```

b. Insert NOPs for correct execution (assuming data forwarding without hazard detection is implemented, and branch comparison happens in 2nd stage). [2 marks]
When branch comparison occurs in the 2nd stage (ID stage), branch inputs must be forwarded to the ID stage:

1. L1-L2 (ALU → Branch in ID):
   L2 needs x14 in its ID stage. The ALU result of the first instruction is ready at the end of its EX stage (cycle 3). Delaying L2 by 1 NOP allows the result to be forwarded from the EX/MEM pipeline register to L2's ID stage in cycle 4. Thus, 1 NOP is required.
2. L3-L4 (Load → Branch in ID):
   L3 (ld) loads x13, which is ready at the end of its MEM stage (cycle 4). L4 (beq) needs x13 in its ID stage. For the loaded value to be forwarded to ID, L4's ID stage must be delayed to cycle 5. Thus, 2 NOPs are required between L3 and L4.
3. L5-L6 & L5-L7:
   Forwarding to EX stage works without stalls. Hence, no NOPs are needed here.

Therefore, the final code is:

```
        add x14, x12, x11
        nop
    L2: beq x15, x14, L1
        ld x13, 8(x13)
        nop
        nop
        beq x12, x13, L2
    L1: and x13, x15, x13
        ld x11, 4(x13)
        sd x13, 0(x15)
```